

Reviewer Pack — Persistent Homology Barcodes and Communication Complexity of...

Entry ceaca8c608a7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 01:44:44 UTC

Persistent Homology Barcodes and Communication Complexity of Boolean Functions

Entry ID: ceaca8c608a7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 01:44:44 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology **Field B** (complexity object): Communication Complexity

Statement:

For any Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the sum of the lengths of the persistent homology bars in its communication matrix's Vietoris-Rips complex satisfies $\sum L_i = \Theta(\log CC(f))$, where $CC(f)$ is the deterministic communication complexity. The barcode lengths L_i are measured in the scale of the input space's diameter.

Rationale (proposer's reasoning):

Persistent homology captures topological features of the communication matrix's geometry, which may reflect the minimal 'information bottleneck' required for coordination between parties. The barcode lengths could quantify how the function's structure resists simplification via partitioning, aligning with KW's protocol cost-to-circuit-size equivalence.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): adbcbf2e2f4c714e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=46.8s

5.1 Generated Python source

```
import random
import math

def generate_random_3cnf(n):
    clauses = []
    for _ in range(2 * n):
        clause = []
        for _ in range(3):
            var = random.randint(0, n - 1)
            sign = random.choice([-1, 1])
            clause.append((sign, var))
        clauses.append(clause)
    return clauses

def communication_matrix(clauses, n):
    m = 2 ** n
    matrix = [[0] * m for _ in range(m)]
    for i in range(m):
        for j in range(m):
            x = [int(bin(i)[2:].zfill(n)[j // (1 << k) & 1]) for k in range(n)]
            y = [int(bin(j)[2:].zfill(n)[k // (1 << k) & 1]) for k in range(n)]
            if all((x[var] == sign or x[var] == -sign) for sign, var in clauses):
                matrix[i][j] = 1
    return matrix

def viterbi_algorithm(matrix, n):
    m = 2 ** n
    dist = [[math.inf] * m for _ in range(m)]
    prev = [[None] * m for _ in range(m)]
    dist[0][0] = 0

    for i in range(m):
        for j in range(m):
            if matrix[i][j] == 1:
                for k in range(n):
                    if (i & (1 << k)) != (j & (1 << k)):
                        new_dist = dist[i][k] + 1
```

```

        if new_dist < dist[j][k]:
            dist[j][k] = new_dist
            prev[j][k] = i

    return dist, prev

def backtrack(prev, n):
    m = 2 ** n
    paths = []
    for j in range(m):
        path = [j]
        k = j
        while prev[k][path[-1]] is not None:
            path.append(prev[k][path[-1]])
            k = path[-1]
        paths.append(path[::-1])
    return paths

def persistent_homology(matrix, n):
    m = 2 ** n
    dist, _ = viterbi_algorithm(matrix, n)
    bars = []

    for i in range(m):
        for j in range(i + 1, m):
            if dist[i][j] < math.inf:
                bars.append((dist[i][j], dist[j][i]))

    return bars

def karchmer_wigderson_protocol_cost(n):
    # Simplified approximation for demonstration purposes
    return n * (n - 1) // 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 30
    clauses = generate_random_3cnf(n)
    matrix = communication_matrix(clauses, n)
    bars = persistent_homology(matrix, n)
    barcode_lengths = [bar[0] for bar in bars]
    sum_barcode_lengths = sum(barcode_lengths) / (2 ** n)
    cc_f = karchmer_wigderson_protocol_cost(n)

    if cc_f == 0:
        return {
            "metric_name": "Sum of Barcode Lengths",
            "metric_value": sum_barcode_lengths,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "CC(f) is zero, cannot compute  $\theta(\log CC(f))$ "
        }

    ratio = sum_barcode_lengths / math.log(cc_f)
    return {
        "metric_name": "Sum of Barcode Lengths",

```

```

        "metric_value": sum_barcode_lengths,
        "instances_tested": 1,
        "conjecture_holds": abs(ratio - 1) < 0.1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std = math.sqrt(sum((r['metric_value'] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['con]
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support/falsification criteria
 | next: Re-run test with debugging enabled to identify crash cause

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	17369
2	preregistration	ollama_remote	qwen3:8b	0	0	20023
3	novelty	ollama_remote	qwen3:8b	0	0	16525
4	novelty	ollama_remote	qwen3:8b	0	0	11191
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14107
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45692
7	judge	ollama_remote	qwen3:8b	0	0	122659

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 247566 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/ceaca8c608a7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ceaca8c608a7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ceaca8c608a7.tar.gz` (if generated)